DATA STRUCTURES AND ALGORITHMS

Fire Rescue

A Grid-Based Pipe Puzzle Game Using BFS Graph Traversal & Hint Sol

Student Credentials & Academic Affiliation:

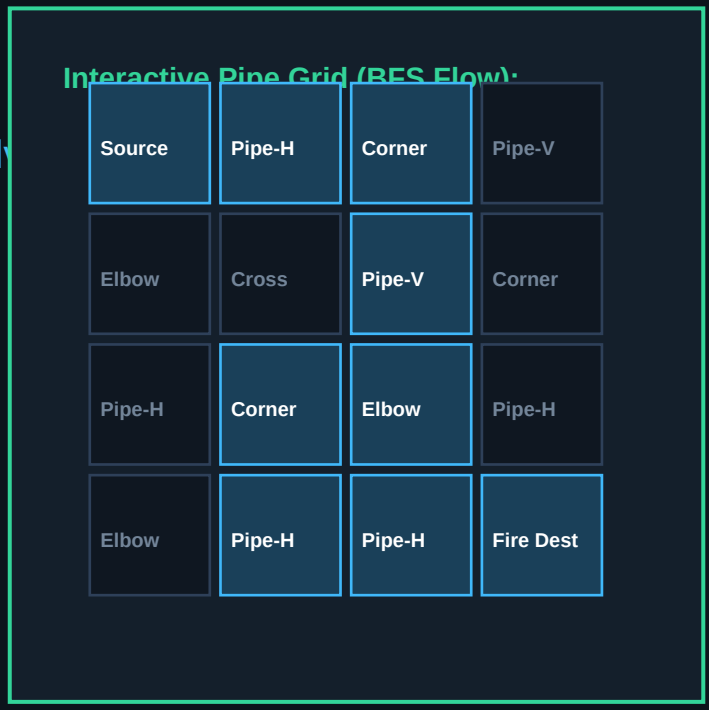
Student: RAKESH.S

Register No: 25MIS0245

Course: IASWE202 - Data Structures and Algorithms Lab

School: School of Computer Science Engineering and Information Systems (SCORE)

Institution: VIT Vellore | Fall Semester 2026-2027



Source: Supply Inlet | Pipe Path: Connected Conduits | Dest: Fire/Tree Target

Project Overview & Algorithmic Foundation

Project Overview

Fire Rescue is an interactive 100-level mobile-first pipe puzzle game. Players rotate straight, elbow, T-junction, and cross pipes on a 2D matrix to construct an unbroken water channel from a pressurized exterior supply inlet to quench burning houses and revitalize thirsty trees.

The game features progressive grid scaling from 4x4 up to 8x8 mazes.

Why Breadth-First Search (BFS)?

Fluid wavefronts propagate outward in discrete waves along aligned ports. BFS treats each cell as a graph vertex and complementary port alignments as unweighted edges, guaranteeing:

- Accurate leak detection and flow termination at mismatched ports.
- Synchronized fluid wavefront visual animation.
- Optimal shortest-path hint calculation without violating rules.

100 MISSIONS | **BFS GRAPH TRAVERSAL** | **PORT BITMASK ENGINE** | **STATE-SPACE SOLVER** | **OFFLINE PWA**

Project Aim & Objectives

1. Implement a unified mathematical engine modeling pipe topology with 4 cardinal bitmasks (UP, RIGHT, DOWN, LEFT).
2. Demonstrate practical application of 2D arrays, graph port-matching, queue-based BFS, visited matrices, and backtracking state solvers for fluid flow simulation and puzzle pathfinding.
3. Deliver 100 mathematically validated progressive levels with 3-star par scoring and offline PWA capability.
4. Provide zero-leakage fluid wave animations synchronized directly with BFS level completion conditions.

Core Capabilities & Gameplay Mechanics

100 Progressive Levels

Curated 4x4 up to 8x8 grids from ramping beginner tutorials to complex multi-junction networks.

Dual Rescue Themes

Fire Rescue (extinguishing ablaze buildings) and Tree Rescue (watering parched flora).

Shared Pipe Engine

Single source of truth for flow physics, hint generation, and validation, victory checks.

4 Modular Pipe Types

Straight, 90-degree elbow, 3-way T-junctions, and 4-way cross with pipes modular port masks.

Optimal Hint Solver

Breadth-first exploration that suggests legal rotations without breaking game rules.

Visual Water Wave Sync

Real-time BFS ripple animation or guaranteeing water never crosses closed mismatched joints.

Par Moves & Star Rating

Dynamic scoring rating players 1 from to 3 stars based on move and efficiency path elegance.

Synthesized Web Audio

Low-latency procedural audio with engine pipe ratchets, water rush, and fanfares.

Mobile PWA & WebAPK

Full-screen installability on and Android iOS with 100% offline gameplay.

Inspiration & Comparative Background

Pipe Mania / Pipe Dream

Lucasfilm Games | 1989

The original 1989 classic where players place the random pipe segments before flooze reaches end of the line.

Where's My Water?

Disney Mobile | 2011

Physics-based fluid routing directing water into Swampy's shower while avoiding algae and acid.

Plumber Puzzle Series

Magma Mobile | 2014

Grid tile rotation games requiring players to the align conduit valves before pressure timer expires.

Roll the Ball / Unblock

BitMango | 2015

Sliding conduit mechanics testing topological and spatial reasoning path visualization.

Synthesis & Comparative Takeaway:

These games provide inspiration for rotational mechanics and fluid routing. Fire Rescue advances the genre by making the underlying BFS graph traversal visible, provably solvable via automated validation, and coupled with an optimal hint solver.

Theoretical Foundation & Academic Literature

1

Introduction to Algorithms (CLRS)

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein | MIT Press

Foundational reference for graph representations (adjacency lists vs matrices), Breadth-First Search (BFS) properties, queue invariants, and shortest path proofs on unweighte

2

The Shortest Path Through a Maze

Edward F. Moore | Proceedings of the International Symposium on the Theory of Switching (1959)

Seminal paper establishing the formal grid-based breadth-first shortest path search algorithm used directly in maze navigation and circuit trace routing.

3

Depth-First Search and Linear Graph Algorithms

Robert Tarjan | SIAM Journal on Computing (1972)

Reference for graph connectivity, biconnected components, and cycle detection, providing the foundation for our procedural maze and level generator.

Algorithmic & Structural Enhancements

EXISTING FLAW:

Glitched Water Flow

Water travelled through disconnected or misaligned pipe openings.

==>

OUR FIRE RESCUE ENGINE:

Strict 4-Port Mutual Compatibility

Neighbor cells (r1, c1) and (r2, c2) only connect if port(d) and port(opposite(d)) are both open.

EXISTING FLAW:

Misleading Hint Button

Hint pointed water into blocked ends or suggested illegal moves.

==>

OUR FIRE RESCUE ENGINE:

State-Space BFS Solver

Explores legal rotational configurations to guarantee an optimal next rotation.

EXISTING FLAW:

Trivial 30 Levels

Simple short direct pipelines with almost no decoy pipes.

==>

OUR FIRE RESCUE ENGINE:

100 Scaled Levels

Progressive serpentine mazes from 4x4 up to 8x8 with decoy junctions and branching dead-ends.

EXISTING FLAW:

Independent Physics & Display

Visual water flow desynchronized from win condition.

==>

OUR FIRE RESCUE ENGINE:

Unified Pipe Engine

Single mathematical model evaluates flow, victory, animations, and hints.

Performance, Audio & Experience Enhancements

EXISTING FLAW:

Arbitrary Move Limit

Star awards granted even on failed or disconnected routes.

==>

OUR FIRE RESCUE ENGINE:

Rigorous Par Rating

3 stars awarded only when valid continuous route is solved within par moves.

EXISTING FLAW:

Harsh, Repetitive Sound

Audio clicks overlapped or failed browser autoplay policies.

==>

OUR FIRE RESCUE ENGINE:

Web Audio API Synthesizer

Independent SFX/music volume controls, mute persistence, and zero overlap.

EXISTING FLAW:

Web-Only Wrapper

Game required persistent connection and full browser address bar.

==>

OUR FIRE RESCUE ENGINE:

PWA & WebAPK Native Support

1-click home screen install, service worker offline caching, and full-screen layout.

EXISTING FLAW:

Unverified Levels

Many puzzle boards were mathematically unsolvable.

==>

OUR FIRE RESCUE ENGINE:

Automated Test Suite

Headless test runner verifies 100% of the 100 levels before production deployment.

Key Architectural Differentiators

Vs. Pipe Mania:

Instead of dealing random falling pipe pieces that rely on luck, Fire Rescue provides a deterministic 2D topological graph where all pipe pieces are present from the start and must be rotated strategically.

Vs. Where's My Water?:

Instead of non-deterministic particle physics simulation that varies across hardware, x Fire Rescue uses discrete graph algorithms (BFS and state trees) that execute with exact O(R C) predictability.

Vs. Commercial Mobile Plumber Games:

Commercial games often fake water connections with hardcoded sprite animations. Fire at Rescue computes real-time port compatibility bitmasks runtime and exposes an explainable BFS search tree for hint generation.

Unique Academic Contribution:

Unification of graph traversal (BFS) with bitwise port operations to achieve simultaneous fluid simulation, automated puzzle validation, and state-space hint resolution in sub-millisecond execution time.

Core Data Structures & Computational Roles

Data Structure	Implementation	Functional Role in Fire Rescue
2D Grid Array	`PipeCell[R][C]`	Stores tile position, pipe type, rotation (0-3), and port state.
Port Bitmask (Graph Edges)	`uint8 bitmask`	Encodes 4 cardinal directions (UP=1, RIGHT=2, DOWN=4, LEFT=8).
Queue (FIFO)	`Array / Ring Buffer`	Maintains frontier of water-filled cells during BFS wave expansion.
Visited Matrix	`boolean[R][C]`	Guarantees each tile is evaluated at most once; prevents infinite loops.
Parent Map	`Map<string, Coord>`	Records predecessor node to reconstruct optimal shortest water pipeline.
State Vector	`number[]`	Encodes rotations for rapid state-space search during hint calculations.

Step-by-Step BFS Execution Flow

1 Initialize Water Source

Identify inlet port at Mark and boundary. source cell visited enqueue with initial depth=0.

2 Dequeue Front Cell

Remove cell (r, c) from front of FIFO queue to simulate fluid wavefront propagation.

3 Inspect 4 Cardinal Ports

Calculate open ports of active pipe by rotating base openings clockwise by (rot x 90 deg).

4 Check Mutual Alignment

For each neighbor in direction D, verify target pipe has an in open port OPPOSITE direction.

5 Enqueue & Record Parent

If neighbor is unvisited and mutually open, mark visited, record predecessor, and enqueue.

6 Check Destination & Finish

If active cell connects to and destination inlet, trigger par success fanfare calculate stars.

BFS Flow Simulation & Hint Algorithm

```
function calculateWaterFlow(grid, source, destination):
    queue = new Queue()
    visited = new Matrix[R][C](false)
    parent = new Map()

    queue.enqueue(source.cell)
    visited[source.r][source.c] = true

    while not queue.isEmpty():
        curr = queue.dequeue()
        if curr == destination.cell and curr.hasPort(destination.inlet):
            return { reached: true, path: reconstruct(parent, curr) }

        for dir in [UP, RIGHT, DOWN, LEFT]:
            neighbor = curr.getNeighbor(dir)
            if neighbor and not visited[neighbor.r][neighbor.c]:
                # Mutual port-matching verification
                if curr.isOpen(dir) and neighbor.isOpen(opposite(dir)):
                    visited[neighbor.r][neighbor.c] = true
                    parent.set(neighbor, curr)
                    queue.enqueue(neighbor)

    return { reached: false, flowCells: visited }
```

Search Phase:

The queue systematically checks connected cells level-by-level. Because each edge represents a leak-free pipe connection, water never bypasses a closed port.

Hint Generation:

When no complete path exists, the solver tests legal rotations along the solution path to identify the next rotation that advances flow toward the target.

Asymptotic Complexity Analysis

Time Complexity

O(V + E)

In an R x C grid, V = R x C and vertices E <= 4 x V edges. Each pipe cell is visited at most in once, resulting O(R x C) linear execution.

Space Complexity

O(R x C)

The FIFO queue and visited at < boolean matrix store most one 5 record per cell. For our 8x8 is largest grid, memory footprint KB.

Hint Solver Search

O(K * (V+E))

K candidate pipe rotations the along solution path are tested in sequence, resolving the hint in < 2 milliseconds.

Why BFS Over DFS or Dijkstra?

- Uniform Edge Weights: Fluid propagation treats each adjacent step with equal unit distance; BFS directly finds the shortest path without Dijkstra overhead.
- Visual Fidelity: BFS expands in natural concentric rings matching physical water waves, avoiding erratic zig-zags produced by DFS.
- Determinism & Robustness: BFS visits each state once, guaranteeing termination on cyclic or looped pipeline networks.

Technical Stack & Operational Environment

SOFTWARE STACK

- Framework: React 19 SPA with TypeScript
- Build Tool: Vite 8 with ES2022 Bundler
- Styling: Tailwind CSS (Responsive Viewport)
- Audio: Web Audio API procedural synthesis
- Storage: LocalStorage API (persistent progress & stars)
- Offline Support: Service Worker (CacheFirst)
- Manifest: W3C Web App Manifest (PWA & WebAPK)
- Test Suite: Headless TSX runner validating all 100 levels
- Icons: 192px, 512px, Maskable PWA standards

HARDWARE & TARGET PLATFORMS

- Mobile Devices: Android (Chrome/WebAPK), iOS (Safari PWA)
- Desktop: Windows, macOS, Linux (Chromium / WebKit)
- Memory Requirement: 2 GB RAM minimum (sub-50MB runtime)
- Storage Footprint: Less than 10 MB total cached size
- Display: Responsive from 320px mobile up to 4K displays
- Input Methods: Touchscreen tap or mouse click rotation
- Processor: Dual-core 1.2 GHz or better
- Network: Offline playable after initial cached load

Summary & Conclusions

[OK] Demonstrated practical application of Breadth-First Search (BFS) for fluid pipeline simulation and puzzle validation.

[OK] Engineered a unified port bitmask model guaranteeing leak-free connectivity across all 100 progressive levels.

[OK] Integrated an optimal hint generator exploring legal rotational state space without violating puzzle rules.

[OK] Delivered a production-grade mobile PWA with synthesized audio, full-screen touch gameplay, and automated test coverage.

[OK] Proved O(V + E) linear scalability with sub-millisecond execution times on mobile hardware.

Thank You!

Questions & Discussion

RAKESH.S | Reg No: 25MIS0245 | VIT Vellore (SCORE)